

Shopease E-Commerce Platform

Comprehensive Viva, Practical Exam & Technical Interview Preparation Guide

30+ In-Depth Questions

Code-Grounded Answers

Full-Stack Node/Express/MongoDB

Security & Cloud DevOps

EXAMINER & INTERVIEW FOCUS

This guide contains the most critical technical questions an external examiner, professor, or technical interviewer will ask about the Shopease codebase. Each answer provides the exact theoretical concept combined with direct code references from this project.

Section 1: Architecture, Design Patterns & Project Structure

Q1 What architectural pattern does Shopease follow and how is the codebase structured?

Answer: Shopease strictly follows the **Model-View-Controller (MVC)** architectural pattern:

- **Model (`src/models/`):** Defines Mongoose schemas, data validation, indices, and business methods (e.g. `User.js`, `Product.js`, `Category.js`, `Order.js`).
- **View (`views/`):** Server-side rendered EJS templates organized into shared layouts (`views/layouts/main.ejs`), reusable partials (`views/partials/navbar.ejs`, `product-card.ejs`), and domain pages (`home.ejs`, `shop.ejs`, `admin/`, `supplier/`).
- **Controller (`src/controllers/`):** Contains request handlers, database queries, and response rendering logic (e.g. `productController.js`, `supplierController.js`, `adminController.js`).

Routes in `src/routes/` decouple URL endpoint declarations from controller logic, and reusable middleware in `src/middleware/` handles cross-cutting concerns (auth guards, session cart calculations, error handling).

Q2 Why did you choose Server-Side Rendering (SSR with EJS) instead of a Single-Page Application (SPA with React/Vue) for this project?

Answer:

1. **Search Engine Optimization (SEO):** E-commerce product catalogs depend heavily on web crawlers (Google, Bing). With SSR, search engines receive fully hydrated semantic HTML on the initial HTTP request without executing client-side JavaScript bundles.
2. **Largest Contentful Paint (LCP) & Fast Initial Load:** The browser renders the HTML and CSS immediately without waiting to download, parse, and execute multi-megabyte JavaScript single-page application bundles.
3. **Architectural Simplicity:** State is maintained securely on the server via authenticated sessions and MongoDB, eliminating the overhead of managing complex client-side state stores (Redux, Pinia) or token refresh cycles.

Q3 How does layout inheritance work with `express-ejs-layouts` ?

Answer: In `server.js`, we configure:

```
app.use(expressLayouts);
app.set('layout', 'layouts/main');
```

The master template `views/layouts/main.ejs` defines the global HTML5 boilerplate, `<head>` meta tags, CSS stylesheets, top banner, global navbar, footer, and the Quick View `<dialog>` container. It contains a placeholder tag `<%- body %>`. When any controller executes `res.render('pages/shop', data)`, Express injects the parsed HTML of `shop.ejs` directly into the `<%- body %>` slot of `main.ejs`.

Section 2: Database Design & Mongoose Modeling

Q4 When did you use embedded subdocuments versus relational references (`ObjectId`) in MongoDB?

Answer:

- **References (Normalized):** We used `{ type: mongoose.Schema.Types.ObjectId, ref: 'User' }` for `product.supplier` and `wishlist` items. Products and Users have independent lifecycles, and a supplier may own thousands of products. Referencing avoids document size bloat and keeps the 16MB BSON document limit intact.
- **Embedded Subdocuments (Denormalized):** We embedded `specs: [{ key, value }]`, `features: [String]`, and `trackingEvents: [{ status, message, timestamp, location }]` directly within products and orders. Specifications and tracking events are always queried together with their parent document and have no independent existence outside it.
- **Hybrid Snapshot Pattern:** In `Order.js`, we store a reference to `product`, but we also embed snapshots of `name`, `price`, and `thumbnail` at purchase time. This ensures that if a merchant updates a product's price from \$100 to \$150 next month, past historical orders remain accurate at \$100.

Q5 Explain how the Mongoose `pre('save')` hook works in `User.js`. What does `this.isModified('password')` prevent?

Answer: In `src/models/User.js`:

```
userSchema.pre('save', async function (next) {
  if (!this.isModified('password')) return next();
  const salt = await bcrypt.genSalt(10);
  this.password = await bcrypt.hash(this.password, salt);
  next();
});
```

Explanation: This is a document middleware hook that intercepts any save operation before it writes to MongoDB. The guard `if (!this.isModified('password')) return next();` is critical: if a user updates their name or shipping address without changing their password, this hook will skip hashing. Without this check, the already-hashed bcrypt string (e.g. `$2a$10$...`) would be re-hashed, permanently corrupting the user's password and locking them out.

Q6 What is the purpose of the `slug` field in Product and Category schemas?

Answer: Instead of exposing raw database IDs in URLs (e.g. `/product/6a98f587d605aafcd96bb0d2`), we generate URL-friendly slug strings (e.g. `/product/aura-studio-wireless-anc-headphones`). In Mongoose, `slug` is defined with `unique: true, lowercase: true, index: true`. This achieves three goals:

1. **SEO:** Search engine crawlers prioritize human-readable keywords in the URL path.
2. **Security:** Prevents sequential ID enumeration attacks.
3. **Query Performance:** The unique index allows MongoDB to perform B-Tree lookups in $O(\log N)$ time rather than scanning all documents.

Q7 How does Mongoose population (`.populate()`) work? Is it an SQL JOIN?

Answer: No, MongoDB is a NoSQL document database and does not perform native relational SQL JOINS across separate tables. When we call:

```
const order = await Order.findById(id).populate('orderItems.product');
```

Mongoose executes an initial query to fetch the Order document, extracts all `product` ObjectIds, and then executes a second query (`Product.find({ _id: { $in: productIds } })`) under the hood. Mongoose then joins the returned product objects into the `orderItems.product` properties in memory before returning the document to the controller.

Section 3: Authentication, Sessions & Security

Q8 Why did you use Session-based authentication with `connect-mongo` instead of stateless JSON Web Tokens (JWT)?

Answer:

1. **Instant Revocation:** With JWTs, once an access token is signed, it is valid until expiration unless a token blacklist is maintained in Redis. With session-based authentication in MongoDB, if an admin disables a compromised account, deleting the session record in MongoDB instantly revokes access on the next HTTP request.
2. **Protection from XSS:** Storing JWTs in browser `localStorage` exposes them to Cross-Site Scripting (XSS) theft. Express sessions use `httpOnly` cookies, which JavaScript running in the browser cannot read.
3. **Cart Integration:** Guest shopping carts and authenticated carts seamlessly bind to the same session identifier (`req.session.cart`).

Q9 Explain the security flags configured on the session cookie in `server.js`.

Answer:

```
cookie: {
  maxAge: 14 * 24 * 60 * 60 * 1000, // 14-day persistent login
  httpOnly: true,                    // Cannot be accessed via document.cookie
  sameSite: 'lax',                  // Prevents Cross-Site Request Forgery (CSRF)
  secure: process.env.NODE_ENV === 'production' // Transmitted over HTTPS only
}
```

- **`httpOnly: true`**: Prevents malicious third-party scripts from stealing the session cookie via `document.cookie`.
- **`sameSite: 'lax'`**: Ensures the browser only sends the cookie for same-site requests and top-level navigation, blocking cross-site CSRF attacks.
- **`secure: true`**: Instructs the browser to only transmit the cookie over encrypted HTTPS channels, preventing packet sniffing on public Wi-Fi.

Q10 Why is `app.set('trust proxy', 1)` mandatory on cloud hosts like Render.com?

Answer: In modern cloud environments (Render, Heroku, AWS, Cloudflare), the public client connects to a reverse proxy over HTTPS. The reverse proxy terminates the TLS encryption and forwards the request to the internal Node.js container over plain HTTP, adding a header `X-Forwarded-Proto: https`.

By default, Express considers internal HTTP connections unencrypted. If `cookie.secure: true` is configured, Express refuses to set the cookie because it believes the connection is insecure. Enabling `app.set('trust proxy', 1)` instructs Express to trust the first proxy hop and inspect the `X-Forwarded-Proto` header. Without this line, session cookies fail completely in production, and users are logged out on every refresh.

Q11 How does Role-Based Access Control (RBAC) prevent unauthorized route access?

Answer: In `src/middleware/auth.js`, we define gatekeepers:

- `ensureAuth`: Checks if `req.session.user` exists. If not, saves `req.originalUrl` to `req.session.returnTo` and redirects to login.
- `ensureAdmin`: Checks `req.session.user.role === 'admin'`. Returns HTTP 403 Forbidden if violated.
- `ensureSupplier`: Checks `req.session.user.role === 'supplier' || req.session.user.role === 'admin'`.

Crucially, in `src/routes/adminRoutes.js` and `supplierRoutes.js`, we scope these middlewares:

```
router.use('/admin', ensureAdmin);
router.use('/supplier', ensureSupplier);
```

This guarantees that no user can bypass authorization, while preventing route leakage.

Q12 How do you prevent horizontal privilege escalation between two competing suppliers?

Answer: In `supplierController.js`, it is not enough to just check if the user is a supplier. We enforce **database-level ownership scoping**:

```
const filter = { _id: req.params.id };
if (req.session.user.role !== 'admin') {
  filter.supplier = req.session.user._id; // Restricts mutation to items owned by this supplier
}
const product = await Product.findOne(filter);
if (!product) return res.status(404).json({ success: false, message: 'Unauthorized or not found' });
```

If Supplier A sends an HTTP request targeting the product ID of Supplier B, the query returns `null` and the mutation is rejected.

Section 4: Inventory Management & Stock Adjustment Controls

Q13 Walk me through the full lifecycle of a 1-click stock adjustment (e.g. clicking `+5`).

Answer:

1. **Client Action:** The user clicks `<button class="btn-stock-adjust plus" data-action="add" data-amount="5">+5</button>` in the inventory table.
2. **Event Delegation (`main.js`):** A delegated click listener intercepts the event, extracts `productId` and `data-endpoint`, gives button scale feedback, and fires an asynchronous `fetch(endpoint, { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify({ action: 'add', amount: 5 }) })`.
3. **Controller Execution (`supplierController.js` / `adminController.js`):** The backend authenticates the session, verifies supplier ownership, increments the stock with `Math.max(0, product.stock + 5)`, saves the document to MongoDB, and returns:

```
{ success: true, productId: "...", newStock: 33, oldStock: 28, change: 5, message: "Added 5 units..." }
```

4. **DOM Hydration:** The client JavaScript updates `#input-stock-${productId}.value = 33`, swaps the badge pill to `● In Stock (33)`, and triggers a toast notification without any full page reload.

Q14 How does the stock adjustment endpoint support both AJAX and traditional HTML form POSTs?

Answer: Through **Content Negotiation**:

```
if (req.xhr || req.headers.accept?.includes('application/json')) {
  return res.json({ success: true, newStock: product.stock, message: msg });
}
res.redirect('/supplier/products?success=' + encodeURIComponent(msg));
```

If the request originates from client-side `fetch()` with `Accept: application/json`, Express responds with JSON. If JavaScript is disabled or a standard HTML `<form>` submits the data, Express redirects back to the catalog with a flash query parameter.

Q15 How would you handle high-concurrency race conditions (e.g. two shoppers purchasing the last unit simultaneously)?

Answer: In MongoDB, reading a stock value into memory, decrementing in JavaScript, and calling `save()` can lead to race conditions if two threads execute in parallel. In high-concurrency production, we use **atomic database operations**:

```
const updatedProduct = await Product.findOneAndUpdate(
  { _id: productId, stock: { $gte: quantity } }, // Atomic condition check
  { $inc: { stock: -quantity } },                // Atomic decrement
  { new: true }
);
if (!updatedProduct) throw new Error('Insufficient stock or sold out');
```

Because MongoDB executes single-document operations atomically, only the first request decrements the stock; the second request fails the `$gte` condition, completely preventing overselling.

Section 5: Frontend Interactivity & Client-Side JavaScript

Q16 How does the Product Quick View feature work without reloading the page?

Answer:

1. In `views/layouts/main.ejs`, we define a native HTML5 dialog: `<dialog id="quickViewModal" class="quick-view-modal">`.
2. When the user clicks the floating "Quick View" button on any product card, `initQuickViewModal()` in `main.js` calls `modal.showModal()` to render a centered modal with an automatic backdrop blur.
3. It issues an HTTP GET request to `/api/products/${productId}/quickview`.
4. The controller responds with a compact JSON document containing all image angles, specifications, features, verified supplier name, and stock count.
5. Client JavaScript dynamically populates the dialog DOM with an interactive image gallery, quantity stepper, and an AJAX "Add to Cart" button.

Q17 What is "Debouncing" and why is it used in the live search bar?

Answer: Debouncing is a rate-limiting technique that delays executing a function until a certain amount of time has elapsed since the last time the event was triggered.

```
let debounceTimer;
searchInput.addEventListener('input', (e) => {
  clearTimeout(debounceTimer);
  debounceTimer = setTimeout(async () => {
    const res = await fetch('/api/search?q=' + encodeURIComponent(query));
    // render dropdown
  }, 250);
});
```

Without debouncing, if a user types "Headphones" (10 keystrokes), the browser would dispatch 10 separate HTTP requests to MongoDB within milliseconds, causing server CPU spikes. With a 250ms debounce, `clearTimeout` cancels previous timers on every keystroke, sending only 1 request when the user pauses typing.

Section 6: Cart, Pricing & Promotions Engine

Q18 How does the shopping cart state work across requests?

Answer: The shopping cart is maintained in the user's session: `req.session.cart = { items: [], totalQty, subtotal, discount, shipping, total, coupon }`. Because Express Session is backed by `connect-mongo`, this cart is persisted in the `sessions` collection in MongoDB Atlas. If the user refreshes the page, switches browser tabs, or signs in, their cart items are preserved. Once the user completes checkout, the session cart is converted into a permanent `Order` document, and `req.session.cart.items` is cleared.

Q19 How does the coupon calculation engine work for `SAVE20` ?

Answer: In `cartController.js`, when a user submits coupon code `SAVE20`:

```
const VALID_COUPONS = {
  SAVE20: { code: 'SAVE20', discount: 20, type: 'percentage' },
  WELCOME10: { code: 'WELCOME10', discount: 10, type: 'fixed' }
};
```

The code is validated and stored in `req.session.cart.coupon`. Then, the global `initCartAndLocals` middleware recalculates totals on every request:

- `subtotal = sum(price * quantity)`
- `discount = (subtotal * 20) / 100`
- `shipping = (subtotal > 50) ? 0 : 9.99` (Free shipping threshold)
- `total = subtotal - discount + shipping`

Section 7: Express Middleware & Error Pipeline

Q20 How does Express recognize Error-Handling middleware? Why does it require 4 arguments?

Answer: In `src/middleware/errorHandler.js`:

```
const errorHandler = (err, req, res, next) => {
  const statusCode = res.statusCode === 200 ? 500 : res.statusCode;
  res.status(statusCode).render('pages/error', { message: err.message });
};
```

Express inspects function parameter length (`fn.length`). Standard middleware has 3 arguments: `(req, res, next)`. When Express sees **exactly 4 arguments** `(err, req, res, next)`, it registers the function in its internal error pipeline. If any upstream controller calls `next(error)` or throws an error, Express bypasses all remaining standard routes and jumps directly to this 4-argument error handler.

Q21 What was the route collision bug between Admin and Supplier routes and how was it solved?

Answer: Originally, `adminRoutes.js` used an unscoped `router.use(ensureAdmin)`. When mounted in `server.js` with `app.use('/', adminRoutes)`, that middleware intercepted every single URL path on the server. When a supplier visited `/supplier`, `ensureAdmin` checked if their role was `admin`, saw it was `supplier`, and returned 403 Forbidden before the request could ever reach `supplierRoutes` !

The Fix: We scoped the middleware to its respective path prefix:

```
// In adminRoutes.js:
router.use('/admin', ensureAdmin);

// In supplierRoutes.js:
router.use('/supplier', ensureSupplier);
```

Now, requests to `/supplier/*` bypass admin middleware completely.

Section 8: DevOps, Cloud Hosting & Production Operations

Q22 Explain the cloud deployment architecture on Render.com and MongoDB Atlas.

Answer:

1. **Continuous Deployment (CD):** Pushing code to the `main` branch on GitHub triggers an automated webhook to Render.com.
2. **Build Phase:** Render reads `render.yaml`, checks the Node.js engine (`>=18.0.0` in `package.json`), installs production dependencies with `npm install`, and executes `npm start`.
3. **Health Check Validation:** Render sends periodic HTTP GET requests to `/health`. Once our server responds with HTTP 200 `{ status: 'ok' }`, Render's load balancer redirects public domain traffic to the new container and shuts down the old container with zero downtime.
4. **Database Layer:** The application connects over an encrypted TLS connection (`mongodb+srv://...`) to a MongoDB Atlas M0 cluster with IP whitelist `0.0.0.0/0`.

Q23 Why was database seeding (`npm run seed`) necessary after initial deployment?

Answer: When deploying to a brand-new MongoDB Atlas cluster, the database contains zero collections. Without seeding, the homepage would show 0 categories and 0 products, and no admin/supplier accounts would exist to log in. Running `npm run seed` executes `src/seeds/seed.js`, which uses Mongoose models to programmatically insert 8 rich categories, 16 multi-image products, initial stock levels, and pre-hashed user credentials (`admin@store.com`, `supplier@apex.com`, `john@example.com`).

SUMMARY OF KEY CONCEPTS TO MENTION IN VIVA

Always highlight: **MVC Separation of Concerns**, **Bcrypt Pre-save Hook**, **Session Security (httpOnly, sameSite, trust proxy)**, **1-Click AJAX Stock Stepper**, **HTML5 <dialog> Quick View**, and **Zero-Downtime Render.com Cloud Deployment**.